

1. Write a program using C/C++/Java to simulate the priority scheduling (pre-emptive as well as non-preemptive approach) and RR, CPU scheduling algorithms. The scenario is: user may input n processes with respective CPU burst time and arrival time (also take the priority number in case of priority scheduling). System will ask the user to select the type of algorithm from the list mentioned above. System should display the waiting time for each process, average waiting time for the whole system, and final execution sequence.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <queue>
using namespace std;

#define INT_MAX 2147483647
#define INT_MIN -2147483648

struct Process {
    int id, arrivalTime, burstTime, priority;
    int completionTime, turnAroundTime, waitingTime, remainingTime;
};

// Round Robin Scheduling
void roundRobin(vector<Process> processes, int quantum) {
    int currentTime = 0;
    queue<Process*> q;

    for (auto& p : processes) {
        p.remainingTime = p.burstTime;
    }

    vector<Process> complete;
    size_t i = 0;
    while (i < processes.size() || !q.empty()) {
        while (i < processes.size() && processes[i].arrivalTime <= currentTime) {
            q.push(&processes[i]);
            i++;
        }
    }
}
```

```
}
```

```
if (q.empty()) {  
    currentTime++;  
    continue;  
}
```

```
Process* current = q.front();  
q.pop();  
if (current->remainingTime > quantum) {  
    current->remainingTime -= quantum;  
    currentTime += quantum;  
    while (i < processes.size() && processes[i].arrivalTime <= currentTime) {  
        q.push(&processes[i]);  
        i++;  
    }  
    q.push(current);  
} else {  
    currentTime += current->remainingTime;  
    current->remainingTime = 0;  
    current->completionTime = currentTime;  
    current->turnAroundTime = current->completionTime - current->arrivalTime;  
    current->waitingTime = current->turnAroundTime - current->burstTime;  
    complete.push_back(*current);  
}  
}
```

```
cout << "Round Robin Scheduling\n";  
cout << "P.No\tArrival Time\tBurst Time\tCompletion Time\tTurn Around Time\tWaiting Time\n";  
for (auto& p : complete) {  
    cout << p.id << "\t\t" << p.arrivalTime << "\t\t" << p.burstTime << "\t\t" << p.completionTime  
        << "\t\t" << p.turnAroundTime << "\t\t\t" << p.waitingTime << endl;  
}
```

```
float total_wt = 0, total_tat = 0;  
for (const auto& p : complete) {
```

```

    total_wt += p.waitingTime;

    total_tat += p.turnAroundTime;
}

cout << "Avg Waiting Time: " << total_wt / complete.size() << endl;
cout << "Avg Turn Around Time: " << total_tat / complete.size() << endl;
}

// Priority Scheduling Non-Preemptive
void priorityNonPreemptive(vector<Process> processes) {
    int currentTime = 0;

    sort(processes.begin(), processes.end(), [](Process a, Process b) { return a.arrivalTime < b.arrivalTime; });
    vector<bool> isCompleted(processes.size(), false);
    vector<Process> complete;

    for (int i = 0; i < processes.size(); i++) {
        int idx = -1, highestPriority = INT_MIN;
        for (int j = 0; j < processes.size(); j++) {
            if (!isCompleted[j] && processes[j].arrivalTime <= currentTime && processes[j].priority > highestPriority) {
                highestPriority = processes[j].priority;
                idx = j;
            }
        }
        if (idx != -1) {
            Process& current = processes[idx];
            currentTime += current.burstTime;
            current.completionTime = currentTime;
            current.turnAroundTime = current.completionTime - current.arrivalTime;
            current.waitingTime = current.turnAroundTime - current.burstTime;
            isCompleted[idx] = true;
            complete.push_back(current);
        } else {
            currentTime++;
        }
    }
}

```

```

cout << "Priority Scheduling Non-Preemptive\n";
cout << "P.No\tArrival Time\tBurst Time\tPriority\tCompletion Time\tTurn Around Time\tWaiting Time\n";
for (auto& p : complete) {
    cout << p.id << "\t\t" << p.arrivalTime << "\t\t" << p.burstTime << "\t\t" << p.priority
        << "\t\t" << p.completionTime << "\t\t" << p.turnAroundTime << "\t\t\t" << p.waitingTime << endl;
}

float total_wt = 0, total_tat = 0;
for (const auto& p : complete) {
    total_wt += p.waitingTime;
    total_tat += p.turnAroundTime;
}

cout << "Avg Waiting Time: " << total_wt / complete.size() << endl;
cout << "Avg Turn Around Time: " << total_tat / complete.size() << endl;
}

// Priority Scheduling Preemptive
void priorityPreemptive(vector<Process> processes) {
    int currentTime = 0;
    vector<Process> complete;
    for (auto& p : processes) p.remainingTime = p.burstTime;
    vector<bool> isCompleted(processes.size(), false);

    while (complete.size() != processes.size()) {
        int idx = -1, highestPriority = INT_MIN;
        for (int i = 0; i < processes.size(); i++) {
            if (!isCompleted[i] && processes[i].arrivalTime <= currentTime && processes[i].priority > highestPriority) {
                highestPriority = processes[i].priority;
                idx = i;
            }
        }
        if (idx != -1) {
            Process& current = processes[idx];
            current.remainingTime--;
            currentTime++;
            if (current.remainingTime == 0) {

```

```

        current.completionTime = currentTime;

        current.turnAroundTime = current.completionTime - current.arrivalTime;

        current.waitingTime = current.turnAroundTime - current.burstTime;

        isCompleted[idx] = true;

        complete.push_back(current);
    }
} else {
    currentTime++;
}
}

cout << "Priority Scheduling Preemptive\n";

cout << "P.No\tArrival Time\tBurst Time\tPriority\tCompletion Time\tTurn Around Time\tWaiting Time\n";
for (auto& p : complete) {
    cout << p.id << "\t\t" << p.arrivalTime << "\t\t" << p.burstTime << "\t\t" << p.priority
        << "\t\t" << p.completionTime << "\t\t" << p.turnAroundTime << "\t\t\t" << p.waitingTime << endl;
}

float total_wt = 0, total_tat = 0;
for (const auto& p : complete) {
    total_wt += p.waitingTime;
    total_tat += p.turnAroundTime;
}

cout << "Avg Waiting Time: " << total_wt / complete.size() << endl;
cout << "Avg Turn Around Time: " << total_tat / complete.size() << endl;
}

```

```

int main() {
    int n, choice, quantum;

    cout << "Enter number of processes: ";

    cin >> n;

    vector<Process> processes(n);

    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;

        cout << "Enter arrival time, burst time, and priority of process " << i + 1 << ": ";
    }
}

```

```

    cin >> processes[i].arrivalTime >> processes[i].burstTime >> processes[i].priority;
}

    cout << "Select scheduling algorithm:\n1. Round Robin\n2. Priority Scheduling (Non-Preemptive)\n3. Priority
Scheduling (Preemptive)\n";

    cin >> choice;

    if (choice == 1) {
        cout << "Enter time quantum: ";

        cin >> quantum;

        roundRobin(processes, quantum);
    } else if (choice == 2) {
        priorityNonPreemptive(processes);
    } else if (choice == 3) {
        priorityPreemptive(processes);
    } else {
        cout << "Invalid choice\n";
    }

    return 0;
}

```

```

ubuntu@ubuntu-VirtualBox: /media/sf_Ubuntu_Files/Lab6$ ./Lab6
Enter number of processes: 4
Enter arrival time, burst time, and priority of process 1: 0 5 0
Enter arrival time, burst time, and priority of process 2: 1 4 0
Enter arrival time, burst time, and priority of process 3: 2 2 0
Enter arrival time, burst time, and priority of process 4: 4 1 0
Select scheduling algorithm:
1. Round Robin
2. Priority Scheduling (Non-Preemptive)
3. Priority Scheduling (Preemptive)
1
Enter time quantum: 2
Round Robin Scheduling

```

P.No	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
3	2	2	6	4	2
4	4	1	9	5	4
2	1	4	11	10	6
1	0	5	12	12	7

```

Avg Waiting Time: 4.75
Avg Turn Around Time: 7.75
ubuntu@ubuntu-VirtualBox: /media/sf_Ubuntu_Files/Lab6$

```

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab6$ ./Lab6
Enter number of processes: 4
Enter arrival time, burst time, and priority of process 1: 0 5 10
Enter arrival time, burst time, and priority of process 2: 1 4 20
Enter arrival time, burst time, and priority of process 3: 2 2 30
Enter arrival time, burst time, and priority of process 4: 4 1 40
Select scheduling algorithm:
1. Round Robin
2. Priority Scheduling (Non-Preemptive)
3. Priority Scheduling (Preemptive)
3
Priority Scheduling Preemptive
P.No    Arrival Time    Burst Time    Priority    Completion Time    Turn Around Time    Waiting Time
3        2                2             30         4                  2                  0
4        4                1             40         5                  1                  0
2        1                4             20         8                  7                  3
1        0                5             10        12                 12                 7
Avg Waiting Time: 2.5
Avg Turn Around Time: 5.5
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab6$

```

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab6$ ./Lab6
Enter number of processes: 3
Enter arrival time, burst time, and priority of process 1: 0 4 3
Enter arrival time, burst time, and priority of process 2: 1 3 2
Enter arrival time, burst time, and priority of process 3: 2 2 1
Select scheduling algorithm:
1. Round Robin
2. Priority Scheduling (Non-Preemptive)
3. Priority Scheduling (Preemptive)
2
Priority Scheduling Non-Preemptive
P.No    Arrival Time    Burst Time    Priority    Completion Time    Turn Around Time    Waiting Time
1        0                4             3          4                  40                  0
2        1                3             2          7                  63                  0
3        2                2             1          9                  75                  0
Avg Waiting Time: 2.66667
Avg Turn Around Time: 5.66667
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab6$

```